TP - Système de Gestion Hospitalière Avancé avec Entity Framework Core

Contexte professionnel

Vous êtes développeur.NET senior chez **HealthTech Solutions**. Après avoir validé vos compétences de base sur le premier projet, votre chef de projet vous confie un système plus complexe : la gestion intégrée des blocs opératoires, du personnel médical et de la planification des interventions chirurgicales.

Ce nouveau système nécessite des fonctionnalités avancées d'EF Core que vous devez maîtriser.

Cahier des charges fonctionnel

Module Bloc Opératoire

- Un bloc opératoire a un numéro, un étage, et un équipement
- Chaque bloc a un statut (disponible, occupé, en maintenance)
- Un bloc peut accueillir plusieurs interventions dans une journée

Module Interventions Chirurgicales

- Une intervention associe un patient, un chirurgien, et un bloc opératoire
- Elle possède une date de planification, une durée estimée
- L'intervention a un statut (planifiée, en cours, terminée, annulée)
- Plusieurs infirmiers peuvent participer à une même intervention

Module Planning et Disponibilités

- Les chirurgiens ont des créneaux de disponibilité
- Le système doit éviter les conflits de planning
- Gestion des annulations et reprogrammations

Exigences techniques globales

- **Framework** : ASP.NET Core 8+ (API REST)
- **ORM** : Entity Framework Core 8+
- **Base de données** : SQLite (pour développement)
- **Architecture** : Clean Architecture
- **Livrables** : Projet fonctionnel + migrations

Étape 1 : Modélisation avec Héritage (TPH)

Challenge

L'hôpital embauche différents types de personnel médical avec des attributs spécifiques. Utilisez l'héritage TPH (Table per Hierarchy).

Entités à créer

- **Staff** (classe de base abstraite)
  - Id, FirstName, LastName, HireDate, Salary, IsActive
  - CreatedAt, UpdatedAt (Shadow Properties)
- **Surgeon** (hérite de Staff)
  - Specialty, LicenseNumber, YearsOfExperience
- **Nurse** (hérite de Staff)
  - DepartmentId, CertificationLevel, ShiftPreference
- **AdministrativeStaff** (hérite de Staff)
  - Function, OfficeNumber

Travail à réaliser

1. Créer les entités avec héritage TPH (https://learn.microsoft.com/fr-fr/ef/ef6/modeling/designer/inheritance/tph)
2. Configurer l'héritage dans DbContext
3. Créer la migration

Questions

- Quelle stratégie TPH utilisez-vous ? Pourquoi ?
- Comment gérez-vous les propriétés communes ?
- Comment requête-t-on un type spécifique ?

Étape 2 : Relations Many-to-Many avec Propriétés

Challenge

Une intervention chirurgicale implique plusieurs infirmiers. Cette relation nécessite des propriétés additionnelles (rôle de l'infirmier pendant l'intervention).

Exigences

- Relation Many-to-Many entre Nurse et Surgery
- Propriétés de liaison : RoleDuringSurgery, IsScrubNurse

Travail à réaliser

1. Créer l'entité de jointure SurgeryNurse
2. Configurer les relations

3. Créer une migration

Questions

- Pourquoi pas de Many-to-Many directe ?
- Comment charger les infirmiers d'une intervention ?

Étape 3 : Owned Types et Value Objects

Challenge

L'adresse et les coordonnées sont utilisées dans plusieurs entités. Utilisez les Owned Types pour factoriser.

Exigences

- Créer le Value Object ContactInfo (Phone, Email, EmergencyContact)
- Utiliser Owned Types pour Patient, Department, OperatingRoom

Travail à réaliser

1. Créer le Value Object ContactInfo
2. Configurer comme Owned Entity
3. Mapper les colonnes dans la table

Questions

- Quelle différence entre Owned Type et Value Object ?
- Comment valider un Owned Type ?

Étape 4 : Soft Delete et Query Filters

Challenge

On ne supprime jamais définitivement les données médicales. Implémentez le Soft Delete.

Exigences

- Toutes les entités principales ont un champ IsDeleted
- Les queries doivent filtrer automatiquement les enregistrements supprimés
- Possibilité de récupérer les données supprimées (pour admins)

Travail à réaliser

1. Ajouter la propriété IsDeleted aux entités
2. Implémenter un Global Query Filter
3. Créer une méthode pour le "vrai" supprimer

Questions

- Comment le Global Query Filter fonctionne-t-il ?

- Comment surcharger le filter pour les admins ?

Étape 5 : Shadow Properties et Conventions

Challenge

Toutes les entités doivent avoir des métadonnées de traçabilité (CreatedBy, UpdatedBy).

Exigences

- Ajout automatique de CreatedAt, UpdatedAt
- Stockage de l'utilisateur via CurrentUserService (Shadow Property)
- Utiliser les conventions Fluent API

Travail à réaliser

1. Configurer les Shadow Properties
2. Implémenter un interceptor pour les remplir automatiquement
3. Créer une migration

Questions

- Qu'est-ce qu'une Shadow Property ?
- Comment définir des valeurs par défaut ?

Étape 6 : Concurrence Optimiste

Challenge

Deux secrétaires modifient simultanément le planning d'une intervention. Le système doit détecter les conflits.

Exigences

- Ajouter RowVersion (concurrency token)
- Gérer DbUpdateConcurrencyException
- Implémenter une stratégie de résolution

Travail à réaliser

1. Configurer le concurrency token
2. Gérer les exceptions de concurrence
3. Tester les conflits

Questions

- Quelle différence entre optimistic et pessimistic locking ?
- Comment informer l'utilisateur du conflit ?

Étape 7 : Requêtes Compilées et Performance

Challenge

Les recherches de disponibilité doivent être rapides malgré le volume de données.

Requêtes à optimiser

- Rechercher les créneaux disponibles d'un chirurgien
- Trouver les interventions planifiées pour une date
- Lister les interventions par bloc opératoire

Exigences

- Utiliser Compiled Queries
- Ajouter les index nécessaires
- Utiliser AsNoTracking() judicieusement

Travail à réaliser

1. Identifier les requêtes critiques
2. Implémenter des compiled queries
3. Ajouter des index via Fluent API

Questions

- Quand utiliser les compiled queries ?
- Quel impact sur la mémoire ?

Étape 8 : Transactions et Unit of Work

Challenge

Créer une intervention implique plusieurs opérations (vérifier disponibilité, créer intervention, notifier).

Exigences

- Implémenter Unit of Work
- Utiliser les transactions explicites
- Gérer le rollback en cas d'erreur

Travail à réaliser

1. Créer l'interface et implémentation Unit of Work
2. Refactorer les repositories
3. Implémenter la création d'intervention transactionnelle

Questions

- Quand utiliser une transaction explicite ?

- Comment gérer les transactions distribuées ?

Critères d'évaluation

Compétence	Indicateur
Héritage	Maîtrise TPH/TPT, choix de stratégie
Relations complexes	Many-to-Many avec propriétés, autoréférences
Features avancées	Owned Types, Shadow Properties, Query Filters
Concurrency	Gestion des conflits, RowVersion
Performance	Compiled queries, index, tracking
Transactions	Unit of Work, gestion d'erreurs

Livrables attendus

1. **Projet ASP.NET Core** fonctionnel
2. **8+ migrations** (une par étape)
3. **Code source** avec commentaires explicatifs
4. **README** avec instructions et diagramme de classes

Questions de synthèse

À rendre dans un fichier `ANALYSE.md` :

1. Quand préférer TPH vs TPT vs TPC ?
2. Comment implémenter un système d'audit complet (qui a modifié quoi, quand) ?
3. Quelle stratégie pour gérer les pics de charge (1000+ interventions/jour) ?
4. Comment tester les transactions et la concurrence ?

Deadline : Fin du cours

Annexe technique : Patterns et Architecture

A. Repository Pattern

Définition

Le Repository Pattern est un pattern de conception qui mediate entre la couche de données et la couche métier. Il isole la logique d'accès aux données derrière des interfaces.

Intérêts dans les projets .NET

Intérêt	Description
Abstraction	Le code métier ne connaît pas EF Core ou SQL
Testabilité	On peut mocker les repositories pour les tests unitaires
Maintenabilité	Un seul endroit pour modifier la logique d'accès aux données
** DRY**	Pas de duplication de queries LINQ dans les services

Implémentation type

```
public interface IPatientRepository
{
    Task<Patient?> GetByIdAsync(int id);
    Task<IEnumerable<Patient>> GetAllAsync();
    Task<Patient> AddAsync(Patient patient);
    Task UpdateAsync(Patient patient);
    Task DeleteAsync(int id);
}

public class PatientRepository : IPatientRepository
{
    private readonly AppDbContext _context;

    public PatientRepository(AppDbContext context)
    {
        _context = context;
    }

    public async Task<Patient?> GetByIdAsync(int id)
    {
        return await _context.Patients.FindAsync(id);
    }
    // ... autres méthodes
}
```

Contreparties

- Peut créer une couche d'abstraction inutile pour des projets simples
- Risque de "repo mania" (un repository par entité)
- Peut masquer la puissance d'EF Core

B. Unit of Work

Définition

Le Unit of Work coordonne plusieurs opérations qui doivent être traitées comme une seule transaction atomique.

Quand l'utiliser ?

Scénario	Unit of Work ?
CRUD simple sur une entité	Non (DbContext suffit)
Opérations multiples liées	Oui
Transactions inter-services	Oui
Audit trail automatique	Oui

Implémentation type

```
public interface IUnitOfWork
{
    IPatientRepository Patients { get; }
    ISurgeryRepository Surgeries { get; }
    IOperatingRoomRepository OperatingRooms { get; }
    Task<int> SaveChangesAsync();
    Task RollbackAsync();
}

public class UnitOfWork : IUnitOfWork
{
    private readonly AppDbContext _context;

    public UnitOfWork(AppDbContext context,
        IPatientRepository patients,
        ISurgeryRepository surgeries,
        IOperatingRoomRepository operatingRooms)
    {
        _context = context;
        Patients = patients;
        Surgeries = surgeries;
        OperatingRooms = operatingRooms;
    }

    public IPatientRepository Patients { get; }
    public ISurgeryRepository Surgeries { get; }
    public IOperatingRoomRepository OperatingRooms { get; }

    public async Task<int> SaveChangesAsync()
    {
        return await _context.SaveChangesAsync();
    }

    public async Task RollbackAsync()
    {
        await _context.Database.RollbackTransactionAsync();
    }
}
```

Utilisation

```
public async Task ScheduleSurgeryAsync(CreateSurgeryDto dto)
{
    using var unitOfWork = _unitOfWorkFactory.Create();

    // Vérifier disponibilité du bloc
    var room = await unitOfWork.OperatingRooms
        .GetAvailableAsync(dto.Date, dto.Duration);

    if (room == null)
        throw new InvalidOperationException("Aucun bloc disponible");

    // Créer l'intervention
    var surgery = new Surgery { ... };
    await unitOfWork.Surgeries.AddAsync(surgery);

    // Sauvegarder atomiquement
    await unitOfWork.SaveChangesAsync();
}
```

C. Dependency Injection

Pourquoi l'utiliser ?

Bénéfice	Explication
Couplage faible	Les dépendances sont injectées, pas créées
Testabilité	On peut injecter des mocks
Flexibilité	Changer d'implémentation facilement
Lifecycle management	Scoped, Transient, Singleton

Dans .NET (ASP.NET Core)

```
// Dans Program.cs
builder.Services.AddScoped<IPatientRepository, PatientRepository>();
builder.Services.AddTransient<ISurgeryService, SurgeryService>();
builder.Services.AddSingleton<IAppSettings, AppSettings>();

// Dans un contrôleur
public class PatientsController : ControllerBase
{
    private readonly IPatientRepository _repository;

    public PatientsController(IPatientRepository repository)
    {
```

```
        _repository = repository;
    }
}
```

Cycles de vie

Service	Quand l'utiliser
Transient	Lightweight, sans état (logger, helpers)
Scoped	Par requête HTTP, DbContext
Singleton	Configuration, caches globaux

D. Clean Architecture

Principe fondamental

> Les dépendances ne peuvent pointer que vers l'intérieur. Les couches internes ne connaissent pas les couches externes.

```
┌──────────────────────────────────┐
│         Presentation             │  ← Controllers, DTOs
├──────────────────────────────────┤
│         Application              │  ← Services, Use Cases
├──────────────────────────────────┤
│           Domain                 │  ← Entities, Value Objects
├──────────────────────────────────┤
│         Infrastructure           │  ← DbContext, Repositories
└──────────────────────────────────┘
```

Mise en place pas à pas

Étape 1 : Créer la structure de solution

```
dotnet new sln -n HospitalManagement
dotnet new classlib -n HospitalManagement.Domain -o
src/HospitalManagement.Domain
dotnet new classlib -n HospitalManagement.Application -o
src/HospitalManagement.Application
dotnet new classlib -n HospitalManagement.Infrastructure -o
src/HospitalManagement.Infrastructure
dotnet new webapi -n HospitalManagement.Api -o src/HospitalManagement.Api
dotnet new xunit -n HospitalManagement.Tests -o
tests/HospitalManagement.Tests

dotnet sln add src/**/*.csproj tests/**/*.csproj
```

Étape 2 : Définir les dépendances

```
Domain ← Application ← Infrastructure ← Api
                    ↑
                Tests
```

```
<!-- Application.csproj -->
<ItemGroup>
  <ProjectReference
Include="..\HospitalManagement.Domain\HospitalManagement.Domain.csproj" />
</ItemGroup>

<!-- Infrastructure.csproj -->
<ItemGroup>
  <ProjectReference
Include="..\HospitalManagement.Domain\HospitalManagement.Domain.csproj" />
  <ProjectReference
Include="..\HospitalManagement.Application\HospitalManagement.Application.
csproj" />
</ItemGroup>

<!-- Api.csproj -->
<ItemGroup>
  <ProjectReference
Include="..\HospitalManagement.Application\HospitalManagement.Application.
csproj" />
  <ProjectReference
Include="..\HospitalManagement.Infrastructure\HospitalManagement.Infrastru
cture.csproj" />
</ItemGroup>
```

Étape 3 : Implémenter chaque couche

Couche Domain (aucune dépendance)

- Entities
- Value Objects
- Interfaces (répositories dans Domain)
- Énumérations
- Exceptions métier

Couche Application (dépend de Domain)

- Services / Use Cases
- DTOs
- Interfaces (services externes)
- Mappers

Couche Infrastructure (implémente les interfaces de Domain/Application)

- DbContext
- Repositories
- Configurations EF
- Services externes

Couche Api (dépend de Application)

- Controllers
- Filters
- Middleware

Étape 4 : Configurer DI dans Api

```
// Program.cs
builder.Services.AddScoped<IPatientRepository, PatientRepository>();
builder.Services.AddScoped<IPatientService, PatientService>();
builder.Services.AddDbContext<AppDbContext>(...);
```

E. Analyse critique : Clean Architecture vs MVC vs DDD

Comparaison des approches

Critère	MVC Classique	Clean Architecture	DDD
Complexité	Faible	Moyenne	Haute
Courbe d'apprentissage	Douce	Moyenne	Raide
Taille projet	Petit-Moyen	Moyen-Grand	Grand
Testabilité	Moyenne	Haute	Haute
Maintenabilité	Dégrade vite	Bonne	Excellente
Domain logic	Mixée avec UI	Isolée	Centrale

Quand choisir quoi ?

Contexte	Architecture recommandée
Petit site web, MVP	MVC Classique
API professionnelle, équipe moyenne	Clean Architecture
Domaine complexe, Microservices	DDD
Startup, évolution incertaine	Commencer MVC, evoluer

Les pièges de Clean Architecture

1. Over-engineering

```
❌ PROBLÈME : 15 projets pour une API de 3 endpoints
✅ SOLUTION : Commencer simple, extraire si besoin
```

2. Anémie du domaine

```
❌ PROBLÈME : Entities sans logique, tout dans les services
✅ SOLUTION : Ajouter des méthodes métier aux entités
```

3. Impact sur la performance

```
❌ PROBLÈME : N+1 queries car "Repository cache"
✅ SOLUTION : Optimiser les queries, utiliser AsNoTracking
```

4. Couplage inadverti

```
❌ PROBLÈME : DTOs dans Domain,violant les couches
✅ SOLUTION : DTOs uniquement dans Application/Api
```

Verdict

La Clean Architecture est un excellent compromis pour les projets de taille moyenne à grande. Elle apporte :

- Une structure claire et prévisible
- Une bonne testabilité
- Une séparation nette des responsabilités

Cependant, elle **n'est pas magique** et ne doit pas remplacer une bonne conception métier. Les pièges mentionnés ci-dessus sont courants et doivent être évités.

Pour des projets plus petits ou des équipes debutantes, **commencer par MVC classique** avec une separation Controllers/Services est souvent preferable. On peut toujours migrer vers Clean Architecture si le projet grandit.

Fin de l'annexe